

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..50

ADVANCED EXAMPLES

 Copy as Markdown

Advanced examples

These examples build on [Writing tests](#) and cover less common needs: separating unit and integration tests, testing business logic directly, multi-user stream interactions, future calls, connection limits, and exception monitoring.

Run unit and integration tests separately

To run unit and integration tests separately, the `"integration"` tag can be used as a filter. See the following examples:

```
# All tests (unit and integration)
dart test

# Only integration tests: add --tags (-t) flag
dart test -t integration

# Only unit tests: add --exclude-tags (-x) flag
dart test -x integration
```

To change the name of this tag, see the `testGroupTagsOverride` configuration option.

Test business logic that depends on `Session`

It is common to break out business logic into modules and keep it separate from the endpoints. If such a module depends on a `Session` object (e.g to interact with the database), then the `withServerpod` helper can still be used and the second `endpoint` argument can be ignored:

```
withServerpod('Given decreasing product quantity when quantity is zero', (
  sessionBuilder,
  _
) {
```



```

var session = sessionBuilder.build();

setUp(() async {
  await Product.db.insertRow(session,
    Product(
      id: 123,
      name: 'Apple',
      quantity: 0,
    ),
  );
});

test('then should throw `InvalidOperationException`',
  () async {
    var future = ProductsBusinessLogic.updateQuantity(
      session,
      id: 123,
      decrease: 1,
    );

    await expectLater(future, throwsA(isA<InvalidOperationException>()));
  });
});

```

Multiple users interacting with a shared stream

For cases where there are multiple users reading from or writing to a stream, such as real-time communication, it can be helpful to validate this behavior in tests.

Given the following simplified endpoint:

```

class CommunicationExampleEndpoint {
  static const sharedStreamName = 'shared-stream';
  Future<void> postNumberToSharedStream(Session session, int number) async {
    await session.messages
      .postMessage(sharedStreamName, SimpleData(num: number));
  }

  Stream<int> listenForNumbersOnSharedStream(Session session) async* {
    var sharedStream =
      session.messages.createStream<SimpleData>(sharedStreamName);

    await for (var message in sharedStream) {
      yield message.num;
    }
  }
}

```

Then a test to verify this behavior can be written as below. Note the call to the `flushEventQueue` helper (exported by the test tools), which ensures that `listenForNumbersOnSharedStream` executes up to its

first `yield` statement before continuing with the test. This guarantees that the stream was registered by Serverpod before messages are posted to it.

```
withServerpod('Given CommunicationExampleEndpoint', (sessionBuilder, endpoints) {
  final userId1 = '550e8400-e29b-41d4-a716-446655440000';
  final userId2 = '6ba7b810-9dad-11d1-80b4-00c04fd430c8';

  test(
    'when calling postNumberToSharedStream and listenForNumbersOnSharedStream '
    'with different sessions then number should be echoed',
    () async {
      var userSession1 = sessionBuilder.copyWith(
        authentication: AuthenticationOverride.authenticationInfo(
          userId1,
          {},
        ),
      );
      var userSession2 = sessionBuilder.copyWith(
        authentication: AuthenticationOverride.authenticationInfo(
          userId2,
          {},
        ),
      );

      var stream =
        endpoints.communicationExample.listenForNumbersOnSharedStream(userSession1);
      // Wait for `listenForNumbersOnSharedStream` to execute up to its
      // `yield` statement before continuing
      await flushEventQueue();

      await endpoints.communicationExample.postNumberToSharedStream(userSession2, 111);
      await endpoints.communicationExample.postNumberToSharedStream(userSession2, 222);

      await expectLater(stream.take(2), emitsInOrder([111, 222]));
    });
});
```

Run a future call

The generated test tools expose your [future calls](#) alongside your endpoints, so a test can invoke one immediately instead of waiting for its scheduled time.

Given a future call class named `ReminderFutureCall` with a `send` method, the accessor is `reminder`, following the same [naming rule](#) as scheduling:

```
withServerpod('Given the reminder future call', (sessionBuilder, endpoints) {
  late Session session;

  setUp(() {
    session = sessionBuilder.build();
  });
```

```
test('when invoked then it records a reminder', () async {
  await endpoints.futureCalls.reminder.send(sessionBuilder, 'user-42');

  final reminders = await Reminder.db.find(session);
  expect(reminders, hasLength(1));
});
});
```

This runs the future call's method directly. It does not exercise scheduling, so use it to test what the call does rather than when it runs.

Too many database connections

Dart's test runner runs test files in parallel, and each `withServerpod` group starts its own server with its own connection pool. On a machine with many cores, enough files running at once can exceed the database's connection limit and fail the run.

This is uncommon, and worth addressing only once you hit it. When you do, raise the limit on the database or cap how many files run at once:

```
dart test -t integration --concurrency=4
```

Testing exception monitoring

The `withServerpod` helper accepts the same `experimentalFeatures` argument as the server, so you can register a [diagnostic event handler](#) in a test and assert that your code reports the exceptions you expect.

Write a handler that records what it receives, so the test can wait for an event:

```
import 'dart:async';

import 'package:serverpod/serverpod.dart';

class TestExceptionHandler extends ExceptionHandler {
  final eventsStreamController =
    StreamController<DiagnosticEventRecord<ExceptionEvent>>();

  Stream<DiagnosticEventRecord<ExceptionEvent>> get events =>
    eventsStreamController.stream;

  @override
  Future<void> handleTypedEvent(
    ExceptionEvent event, {
    required OriginSpace space,
    required DiagnosticEventContext context,
  }) async {
    eventsStreamController.add(DiagnosticEventRecord(event, space, context));
  }
}
```

```
}  
}
```

Then register it for the test run:

```
void main() {  
    var exceptionHandler = TestExceptionHandler();  
  
    withServerpod(  
        'Given withServerpod with a diagnostic event handler',  
        experimentalFeatures: ExperimentalFeatures(  
            diagnosticEventHandlers: [exceptionHandler],  
        ),  
        (sessionBuilder, endpoints) {  
            test(  
                'when calling an endpoint method that submits an exception event '  
                'then the diagnostic event handler gets called', () async {  
                    final result = await endpoints.order.placeOrder(sessionBuilder);  
                    expect(result, 'success');  
  
                    final record =  
                        await exceptionHandler.events.first.timeout(Duration(seconds: 1));  
                    expect(record.event.exception, isA<Exception>());  
                    expect(record.space, equals(OriginSpace.application));  
                    expect(record.context, isA<DiagnosticEventContext>());  
                    expect(  
                        record.context.toJson(),  
                        allOf([  
                            containsPair('serverId', 'default'),  
                            containsPair('serverRunMode', 'test'),  
                            containsPair('serverName', 'Server default'),  
                        ]),  
                    );  
                });  
            },  
        );  
    }  
}
```

Handlers run asynchronously and are not awaited by the code that triggers them, so wait for the event rather than asserting immediately after the call.

Related

- [Writing tests](#): the pieces these examples build on.
- [Configuration](#): the options used above, such as `experimentalFeatures` and `rollbackDatabase`.
- [Best practices](#): conventions worth following.